



MANUAL TÉCNICO

Dessert Sacré — Plataforma Digital para Panaderías

Fecha: 2025

Proyecto:
Dessert Sacré

Programa:
Técnico en desarrollo de software

SENA Centro de biotecnología industrial

Repositorio:
<https://github.com/Shalomoreno>

Integrantes:

Moreno S. Salome
González R. Laura S.
Benavides C. Maria Angelica
Torres M. Laura S.

Palmira, Valle del Cauca — Colombia

1. INTRODUCCIÓN AL PROYECTO

1.1 Contexto y Motivación

Dessert Sacré nació como respuesta directa a una problemática directa en Palmira, Valle del Cauca, donde existe una fuerte cultura panadera a lo largo de décadas. Sin embargo, estas panaderías y reposterías operan en barrios y la gran mayoría son invisibles en el mercado digital. Estos negocios no suelen aparecer en buscadores, no tienen redes sociales, y administran sus pedidos de manera manual en cuadernos, por WhatsApp o de memoria. Se creó una plataforma web desarrollada con Flask (Python) y PostgreSQL diseñada para ofrecer a panaderías y reposterías de Palmira y sus municipios vecinos mediante el aplicativo una forma de gestionar sus pedidos de forma ordenada en conjunto de un sistemas de ventas en línea con un amplio catálogo de productos según el cliente lo necesite.

1.2 Validación de Mercado

El equipo realizó unas entrevistas a 12 panaderías de Palmira, desde negocios de barrio hasta los más reconocidos de la ciudad. Los resultados que se recolectaron fueron que 8 de las 12 panaderías entrevistadas mostraron interés real en implementar la solución planteada anteriormente, y más de la mitad reconoció la necesidad de una herramienta digital como la que plantea Dessert Sacré. Esta investigación sirvió para tomar decisiones de diseño y del desarrollo del aplicativo para que este quedara más acorde a las necesidades recolectadas.

1.3 Modelo de Negocio

Ofrecemos a las panaderías una página web o aplicación que les permite digitalizar su negocio, mostrar sus productos, gestionar pedidos y aumentar sus ventas. Resolvemos problemas como la falta de presencia digital, dificultad para organizar pedidos y poca visibilidad en el mercado. Ofrecemos un servicio tecnológico accesible, personalizable y fácil de usar. Satisfacemos la necesidad de modernización, organización y crecimiento del negocio.

1.4 Alcance del Manual

El presente Manual Técnico describe la arquitectura, el funcionamiento interno, la estructura del código fuente, la configuración del entorno de despliegue con Docker, el esquema de base de datos, la lógica de los módulos principales de la aplicación, las rutas de la API. Está dirigido al personal técnico o desarrolladores encargados del mantenimiento, actualización o implementación del sistema.

2. Arquitectura del Sistema

2.1 Visión General

Dessert Sacré sigue una arquitectura de aplicación web que es desplegada mediante contenedores Docker. Este proyecto se compone de tres capas siendo estas la capa de presentación, es decir el frontend utilizado tales como plantillas HTML/Jinja2 + CSS + JavaScript, la capa de lógica del aplicativo o sea Flask en Python y capa de base datos (PostgreSQL).

2.2 Diagrama de Componentes

Cliente (Navegador Web)

El usuario puede acceder a la aplicación a través de cualquier navegador. Estas páginas son renderizadas mediante el motor de plantillas Jinja2 y servidas como HTML. Además, el carrito de compras opera mediante sesiones del lado del servidor, gestionadas por Flask con cookies firmadas.

Servidor de Aplicación — Flask

El servidor de aplicación es Flask, que actúa como el núcleo de la aplicación, este gestiona el enrutamiento HTTP, la lógica del aplicativo, la autenticación de usuarios, el manejo de sesiones, el envío de correos electrónicos SMTP y la comunicación con la base de datos mediante psycopg2.

Base de Datos — PostgreSQL 15

La base de datos funciona a través de PostgreSQL, está almacena todos los datos de usuarios registrados y los códigos de verificación y recuperación de contraseña, así mismo la base de datos guarda información como pedidos, productos y categorías.

Infraestructura — Docker Compose

Por parte de la infraestructura, la aplicación y la base de datos se despliegan como servicios independientes por medio de Docker Compose. Este servicio web expone el puerto 5000 y el servicio 'db' (PostgreSQL 15) persiste sus datos en un volumen Docker nombrado.

2.3 Flujo de Petición HTTP

En el flujo de petición HTTP primeramente el navegador del usuario envía una petición HTTP al servidor Flask en el puerto 5000. A lo que Flask identifica la ruta (endpoint) correspondiente mediante su sistema de enrutamiento (es decir el `@app.route`), luego se valida la sesión para que se haga consulta en la base de datos mediante psycopg2, ahí se procesan los datos y prepara el contexto de renderizado, donde Jinja2 renderiza la plantilla HTML correspondiente, inyectando el contexto generado en el paso anterior y finalmente Flask devuelve la respuesta HTTP al cliente (HTML renderizado, JSON para las rutas de API, o una redirección HTTP).

2.4 Patrones de Diseño Aplicados

Patrón	Aplicación en Dessert Sacré
MVC (Model-View-Controller)	Separación entre lógica (app.py), plantillas (templates/) y datos (PostgreSQL)
Repository-like (parcial)	Función <code>get_db_connection()</code> centraliza el acceso a la BD
Session-based Auth	Autenticación y autorización mediante sesiones del lado del servidor
Factory Function	<code>crear_tabla()</code> inicializa el esquema de BD al arrancar la aplicación
Decorator Routing	<code>@app.route</code> define los endpoints de la aplicación

3. Stack Tecnológico

3.1 Dependencias del Proyecto

El archivo requirements.txt define todas las dependencias de Python con sus versiones:

Librería	Versión	Propósito
Flask	3.1.2	Framework web principal. Enrutamiento, contexto de solicitud, sesiones y plantillas.
psycopg2-binary	2.9.11	Driver PostgreSQL para Python. Permite ejecutar consultas SQL y gestionar conexiones.
Werkzeug	3.1.3	Utilidades HTTP subyacentes de Flask. Proporciona generate_password_hash y check_password_hash.
Jinja2	3.1.6	Motor de plantillas HTML. Permite lógica de presentación en los archivos .html.
itsdangerous	2.2.0	Firma criptográfica de cookies de sesión. Garantiza la integridad de los datos de sesión.
click	8.3.0	Dependencia de Flask para la interfaz de línea de comandos.
blinker	1.9.0	Sistema de señales de Flask. Habilita el uso de señales de ciclo de vida.
MarkupSafe	3.0.3	Escape seguro de HTML en plantillas Jinja2 para prevenir XSS.
colorama	0.4.6	Soporte de colores ANSI en terminal para logs de Flask en Windows.

3.2 Infraestructura y Entorno

Componente	Versión	Rol
Python	3.13	Entorno de ejecución del servidor Flask
PostgreSQL	15	Sistema de gestión de base de datos relacional
Docker	Compatible con Compose v3.9	Contenedorización del entorno de ejecución
Docker Compose	v3.9	Orquestación multi-contenedor (web + db)
SMTP Gmail	Puerto 587 / STARTTLS	Servicio de envío de correos transaccionales

3.3 Lenguajes y Tecnologías del Frontend

- HTML5 con motor de plantillas Jinja2 para el renderizado en el servidor.
- CSS3 (hojas de estilo propias en el directorio static/).
- JavaScript del lado del cliente para interacciones dinámicas: manejo del carrito vía fetch() a las rutas /api/cart, actualización de cantidades y eliminación de ítems sin recarga completa de página.
- No se utiliza ningún framework de frontend (React, Vue, Angular). La aplicación es server-side rendered (SSR) con enriquecimiento JavaScript mínimo.
- Uso de bootstrap-5.3.8 para la creación e interacción de la barra de navegación y el uso de responsive.

4. Estructura del Repositorio

4.1 Árbol de Directorios Principal

```
dessert-sacre/
├── app.py                # Punto de entrada. Rutas, lógica,
configuración Flask
├── config.py            # Clase Config con variables de entorno y
credenciales
├── requirements.txt      # Dependencias Python con versiones fijadas
├── Dockerfile           # Imagen Docker del servicio web
├── docker-compose.yml   # Orquestación multi-contenedor
├── verificar_conex.py    # Script utilitario para validar conexión a
PostgreSQL
├── reset_code           # Archivo de configuración/reset (entorno
específico)
├── templates/           # Plantillas Jinja2 HTML
│   ├── index.html       # Página raíz (landing público)
│   ├── inicio.html      # Inicio para usuarios no autenticados
│   ├── inicio1.html     # Inicio para usuarios autenticados
│   ├── register.html    # Formulario de registro de usuarios
│   ├── login.html       # Formulario de inicio de sesión
│   ├── verify.html      # Verificación de correo electrónico
│   ├── forgot.html      # Recuperación de contraseña – paso 1
│   ├── reset_code.html  # Recuperación de contraseña – paso 2
│   ├── reset_password.html # Recuperación de contraseña – paso 3
│   └── carrito.html     # Vista del carrito (usuario no
autenticado)
│   ├── carrito1.html    # Vista del carrito (usuario autenticado)
│   ├── pasarela.html    # Pasarela de pagos
│   ├── perfil.html      # Perfil del usuario
│   ├── pedidos1.html    # Historial de pedidos del usuario
│   ├── menu.html        # Menú público
│   ├── menu1.html       # Menú autenticado
│   ├── panaderia.html   # Sección panadería (público)
│   ├── panaderia1.html  # Sección panadería (autenticado)
│   ├── pasteleria.html  # Sección pastelería (público)
│   ├── pasteleria1.html # Sección pastelería (autenticado)
│   ├── reposteria.html  # Sección repostería (público)
│   ├── reposteria1.html # Sección repostería (autenticado)
│   ├── bebidas.html     # Sección bebidas (público)
│   ├── bebidas1.html    # Sección bebidas (autenticado)
│   ├── sobrenosotros.html # Sobre nosotros (público)
│   ├── sobrenosotros1.html # Sobre nosotros (autenticado)
│   ├── redes.html       # Redes sociales (público)
│   ├── redes1.html      # Redes sociales (autenticado)
│   ├── zona_protegida.html # Zona con modal de autenticación
│   ├── dashboard.html   # Dashboard del usuario
│   └── admin/
│       └── dashboard.html # Panel de administración principal
```

```

├── pedidos.html    # Vista de pedidos en panel admin
├── usuarios.html  # Vista de usuarios en panel admin
└── static/        # Archivos estáticos
    ├── css/       # Hojas de estilo CSS propias
    ├── js/        # Scripts JavaScript del cliente
    └── img/        # Imágenes y recursos gráficos

```

4.2 Archivos de Configuración Clave

config.py

Define la clase Config que centraliza todas las configuraciones de la aplicación. Lee las variables de entorno con `os.environ.get()` y usa valores por defecto para desarrollo local. Contiene: `SECRET_KEY` para la firma de sesiones, `DB_CONFIG`

G con los parámetros de conexión a PostgreSQL, `EMAIL_USER` y `EMAIL_PASS` con las credenciales SMTP de Gmail.

docker-compose.yml

Define dos servicios: 'web' (la aplicación Flask) construida desde el Dockerfile local, y 'db' (PostgreSQL 15 desde imagen oficial). El servicio 'web' depende del servicio 'db' mediante `depends_on`, expone el puerto 5000 al host, y monta el directorio actual en `/app` del contenedor para desarrollo con recarga en caliente. La base de datos persiste en el volumen Docker 'postgres_data'.

Dockerfile

Define la imagen Docker del servicio web. Típicamente: imagen base de Python, copia de `requirements.txt` e instalación de dependencias con pip, copia del código fuente, exposición del puerto 5000 y comando de arranque de la aplicación Flask.

5. Esquema de Base de Datos

5.1 Tablas Implementadas en la base de datos

Principalmente usamos pgAdmin5 para la creación e implementación de las siguientes tablas por medio de la función `crear_tabla()` en [app.py](#) inicia la base de datos al arrancar el aplicativo como si las tablas no existieran aún con (CREATE TABLE IF NOT EXISTS).

A continuación el aplicativo cuenta con dos tipos de tablas: registro y recuperación. La primera para el ingreso de los datos de los clientes y la segunda los métodos de recuperación de cuenta en caso del que el usuario olvide su contraseña de acceso.

5.1.1 Tabla: registro

Almacena los datos de todos los usuarios registrados en la plataforma.

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PRIMARY KEY	Identificador único autoincremental del usuario
primer_nombre	VARCHAR(100)	NOT NULL	Primer nombre del usuario
segundo_nombre	VARCHAR(100)	NULL permitido	Segundo nombre del usuario (opcional)
primer_apellido	VARCHAR(100)	NOT NULL	Primer apellido del usuario
segundo_apellido	VARCHAR(100)	NULL permitido	Segundo apellido del usuario (opcional)
correo	VARCHAR(150)	UNIQUE, NOT NULL	Correo electrónico. Llave natural y campo de autenticación
password	TEXT	NOT NULL	Hash de la contraseña generado con Werkzeug (PBKDF2)
telefono	VARCHAR(20)	NULL permitido	Número de teléfono de contacto del usuario
direccion	TEXT	NULL permitido	Dirección de entrega o domicilio del usuario

Columna	Tipo	Restricciones	Descripción
codigo_verificacion	VARCHAR(6)	NULL permitido	Código de 6 dígitos para verificación del correo
verificado	BOOLEAN	DEFAULT FALSE	Indica si el usuario ha verificado su correo electrónico

5.1.2 Tabla: recuperacion

Almacena los códigos temporales de recuperación de contraseña.

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PRIMARY KEY	Identificador único del registro de recuperación
correo	VARCHAR(150)	REFERENCES registro(correo)	Llave foránea al correo del usuario en la tabla registro
codigo	VARCHAR(6)	NOT NULL	Código de 6 dígitos generado aleatoriamente
expiracion	TIMESTAMP	DEFAULT NOW() + 15 min	Marca de tiempo de expiración del código (15 minutos)
usado	BOOLEAN	DEFAULT FALSE	Indica si el código ya fue consumido

5.2 Pedidos

En la versión actual, los pedidos se almacenan en el diccionario en memoria `pedidos_guardados={}` dentro del proceso Flask. Esto significa que los pedidos se pierden al reiniciar el servidor y no son accesibles desde múltiples instancias del proceso. Para una versión productiva, se debe implementar una tabla 'pedidos' en PostgreSQL con la siguiente estructura sugerida:

En la versión actual del aplicativo, los pedidos que ingresan por medio de este se almacenan en el diccionario

Columna	Tipo	Descripción
id	SERIAL PRIMARY KEY	Identificador único del pedido
referencia	VARCHAR(20) UNIQUE NOT NULL	Código de referencia del pedido (ej: PED-AB12CD34)
correo_usuario	VARCHAR(150) REFERENCES registro(correo)	Correo del usuario que realizó el pedido
total	NUMERIC(12,2) NOT NULL	Monto total del pedido
metodo_pago	VARCHAR(50)	Método de pago seleccionado (nequi, banco, efecty)
fecha	TIMEST	Fecha y hora de creación del pedido
estado	VARCHAR(30) DEFAULT 'pendiente'	Estado del pedido (pendiente, confirmado, entregado)

5.3 Seguridad en la Capa de Datos

- Las contraseñas que entran a la base de datos no aparecen como las ingreso el cliente, si no que se cifran con generate_password_hash() de Werkzeug, que aplica el algoritmo ccon salt aleatoria.
- La verificación de contraseñas se realiza con check_password_hash(), que compara de forma segura sin exponer el hash.
- Los códigos de verificación y recuperación son de 6 dígitos generados con random.randint() que se envían al correo que el cliente ingresa.

6. Módulos y Rutas de la Aplicación

6.1 Módulo de Registro de Usuarios

Para el módulo de registro de usuarios se sigue el siguiente flujo de datos que consiste principalmente de dos pasos que son la captura de datos y la verificación del correo electrónico.

Ruta	Método	Función	Descripción
/register	GET	register()	Renderiza el formulario de registro. Limpia sesiones previas de verificación.
/guardar	POST	guardar()	Procesa el formulario: valida campos, verifica correo duplicado, hashea password, inserta usuario, envía código.
/verify	GET/POST	verify()	GET: renderiza el formulario de código. POST: verifica el código y marca al usuario como verificado.
/reenviar_codigo	GET	reenviar_codigo()	Genera y envía un nuevo código. Límite: 3 reenvíos por sesión.

Lógica clave de guardar()

- Verifica que primer_nombre, primer_apellido, correo y password estén presentes.
- Consulta si el correo ya existe en la tabla registro (previene duplicados).
- Aplica generate_password_hash() al password antes de insertar.
- Genera un código de 6 dígitos con random.randint(100000, 999999).
- Inserta el usuario con verificado=FALSE y llama a enviar_codigo().
- Guarda session['correo_verificacion'] y redirige a /verify.

6.2 Módulo de Autenticación

Ruta	Método	Función	Descripción
/login	GET/POST	login()	GET: renderiza login. POST: verifica credenciales, distingue admin vs usuario, gestiona sesión.
/logout	GET	logout()	Limpia toda la sesión con session.clear() y redirige a la raíz.

Ruta	Método	Función	Descripción
/dashboard	GET	dashboard()	Vista del dashboard del usuario. Requiere sesión activa.

Credenciales de Administrador

El administrador se autentica con credenciales hardcodeadas en app.py (ADMIN_EMAIL y ADMIN_PASSWORD). Al autenticarse, se establece `session['admin'] = True` y `session['usuario'] = 'Administrador'`. Para producción, estas credenciales deben migrarse a variables de entorno y la contraseña debe estar hasheada.

6.3 Módulo de Carrito de Compras

El carrito opera completamente mediante la sesión Flask. Tres funciones privadas encapsulan la lógica:

- `_get_cart()`: retorna `session.get('carrito', [])` — la lista de items del carrito.
- `_save_cart(cart)`: guarda la lista actualizada en `session['carrito']`.
- `_get_cart_info(cart)`: calcula total de items y precio total para respuestas de la API.

Ruta	Método	Descripción
/agregar_carrito	POST	Agrega un producto al carrito. Si ya existe, incrementa la cantidad (qty).
/eliminar_del_carrito	GET	Elimina un item por índice del carrito.
/actualizar_carrito	POST	Actualiza cantidades de todos los items del carrito (versión público).
/actualiza_carrito	POST	Igual a la anterior pero redirige a /carritoU (versión autenticada).
/carrito	GET	Renderiza la vista del carrito para usuarios no autenticados.
/carritoU	GET	Renderiza la vista del carrito para usuarios autenticados.
/api/cart	GET	API REST: retorna JSON con items, total_items y total_price del carrito.
/api/cart/remove	POST	API REST: elimina un item por índice. Retorna el carrito actualizado.

Ruta	Método	Descripción
/api/cart/update	POST	API REST: actualiza la cantidad de un item. Retorna el carrito actualizado.

6.4 Módulo de Pasarela de Pagos

La pasarela de pagos es una implementación de pago manual (no integra ninguna API de pagos). Presenta al usuario los datos bancarios para realizar la transferencia o depósito. Los métodos disponibles son: Nequi, Bancolombia (transferencia bancaria) y Efecty.

Ruta	Método	Descripción
/pasarela	GET	Renderiza la página de pasarela. Requiere sesión activa. Muestra carrito y total.
/api/datos-pago	GET	Retorna en JSON los datos bancarios del método seleccionado (parámetro: ?metodo=nequi banco efecty).
/api/crear-pedido	POST	Genera un pedido con referencia única (PED-XXXXXXX), lo guarda en pedidos_guardados y vacía el carrito.
/confirmacion	GET	Confirma la compra, vacía el carrito y redirige a /pasarela.

6.5 Módulo de Recuperación de Contraseña

El flujo de recuperación de contraseña consta de tres pasos, cada uno con su propia ruta:

1. El usuario ingresa su correo en /forgot. Se verifica que exista en la BD, se genera un código de 6 dígitos con expiración de 15 minutos, se inserta en la tabla recuperacion y se envía al correo.
2. El usuario ingresa el código recibido en /reset-code. Se verifica que el código coincida, no haya sido usado y no esté expirado. Si es válido, se marca como usado.
3. El usuario ingresa la nueva contraseña en /reset-password. Se valida que ambos campos coincidan, se hashea la nueva contraseña y se actualiza en la tabla registro.

6.6 Módulo de Administración

El módulo de administración permite la gestión completa del sistema, incluyendo usuarios, pedidos, calificaciones, mensajes y generación de reportes. Todas las rutas están protegidas mediante validación de sesión de administrador (session['admin']).

Ruta	Método	Descripción
/admin	GET	Dashboard principal de administración. Muestra todos los usuarios y un resumen de ventas.
/admin/pedidos	GET	Muestra todos los pedidos registrados junto con el total de ventas acumulado
/admin/usuarios	GET	Muestra todos los pedidos registrados junto con el total de ventas acumulado
/admin/calificaciones	GET	Muestra todas las calificaciones realizadas por los usuarios.
/admin/mensajes	GET	Lista todos los mensajes enviados por los usuarios, ordenados por fecha.
/admin/mensaje/<id>	GET	Permite visualizar el detalle de un mensaje específico
/admin/generar_reporte	GET	Genera un reporte en PDF según el tipo seleccionado (usuarios, mensajes o calificaciones)

Lógica clave de admin()

- Valida que el usuario tenga permisos de administrador mediante session.get("admin").
- Establece conexión con la base de datos
- Consulta todos los usuarios desde la tabla registro. Consulta todos los mensajes desde la tabla mensajes ordenados por fecha descendente
- Calcula el total de ventas sumando los valores de pedidos_guardados.
- Renderiza la plantilla "admin/dashboard.html" enviando toda la información al frontend.

Lógica clave de admin_pedidos()

- Verifica permisos de administrador.
- Calcula el total de ventas a partir de los pedidos almacenados en memoria
- Renderiza la vista "admin/pedidos.html" con la lista de pedidos y el total.

Lógica clave de usuarios_admin()

- Verifica permisos de administrador
- Realiza una consulta SQL para obtener todos los usuarios registrados.
- Cierra la conexión a la base de datos
- Renderiza la vista "admin/usuarios.html" con los datos obtenidos

Lógica clave de admin_mensajes()

- Verifica permisos de administrador.
- Consulta todos los mensajes desde la base de datos ordenados por fecha.
- Retorna la vista "admin/mensajes.html" con la información.

Lógica clave de ver_mensaje(id)

- Verifica permisos de administrador.
- Consulta un mensaje específico utilizando su ID.
- Retorna la vista "admin/ver_mensaje.html" con el detalle del mensaje.

Lógica clave de generar_reporte()

- Verifica permisos de administrador.
- Recibe el tipo de reporte desde el formulario (usuarios, mensajes, calificaciones).
- Genera un documento PDF utilizando la librería ReportLab.
- Consulta la información correspondiente según el tipo de reporte:
- Usuarios: obtiene datos de la tabla registro.
- Mensajes: obtiene datos de la tabla mensajes.
- Calificaciones: usa la lista en memoria.
- Construye dinámicamente el contenido del PDF.
- Retorna el archivo como descarga con nombre "reporte_<tipo>.pdf".

Control de acceso

Todas las rutas del módulo validan la sesión de administrador. En caso de no estar autenticado, el usuario es redirigido al login.

7. Seguridad

7.1 Medidas de Seguridad Implementadas

Medida	Implementación	Alcance
Hash de contraseñas	Werkzeug PBKDF2 (generate_password_hash / check_password_hash)	Las contraseñas nunca se guardan en texto plano, si no que se cifran
Consultas parametrizadas	Placeholders %s en psycopg2	Prevención de inyección SQL en todas las consultas a la BD.
Sesiones firmadas	SECRET_KEY de Flask + itsdangerous	Las cookies de sesión están firmadas criptográficamente y cualquier modificación las invalida.
Verificación de correo	Código de 6 dígitos por email antes de activar cuenta	Previene registros con correos falsos o inaccesibles.
Control de acceso por rol	Verificación de la sesión ['admin'] en rutas de administración.	Separación de privilegios entre usuarios normales y administradores.
Marcado de códigos como usados	Campo 'usado=TRUE' en tabla recuperacion tras consumir el código	Previene la reutilización de códigos de recuperación.
Límite de reenvíos	Máximo 3 reenvíos del código de verificación por sesión	Reducir el abuso del servicio de correo
Escape automático de HTML	MarkupSafe en Jinja2	Prevención de Cross-Site Scripting (XSS) en plantillas.

7.2 Vulnerabilidades Identificadas y Recomendaciones

Credenciales en Código Fuente

Las credenciales de administrador ADMIN_EMAIL y ADMIN_PASSWORD, y las credenciales SMTP están expuestas en app.py y config.py respectivamente. El archivo config.py también contiene la contraseña de la base de datos. Esto expone las credenciales si el repositorio es público.

Recomendación: Migrar TODAS las credenciales a variables de entorno. Usar un archivo .env y la librería python-dotenv para su carga. Nunca incluir credenciales en el código fuente ni en el historial de git.

Contraseña de Admin en Texto Plano

La contraseña del administrador se compara directamente con el valor expuesto en el código fuente en ADMIN_PASSWORD sin hashing.

8. Sistema de Correo Electrónico

8.1 Configuración SMTP

La aplicación dessert sacré utiliza el servicio SMTP de Gmail para el envío de correos transaccionales, es decir el envío de información por medio de un correo único adquirido por la aplicación. Esta conexión se establece en el puerto 587 con STARTTLS, lo que cifra la comunicación con el servidor de correo de Google.

Parámetro	Valor	Descripción
Servidor SMTP	smtp.gmail.com	Servidor de correo saliente de Google
Puerto	587	Puerto estándar para STARTTLS
Seguridad	STARTTLS	Cifrado de la conexión SMTP
Autenticación	Contraseña de aplicación Gmail	No se usa la contraseña de la cuenta, sino una clave de app generada en Google Account
Remitente	dessertsacre@gmail.com	Correo configurado en Config.EMAIL_USER

8.2 Función enviar_codigo()

La función enviar_codigo(correo_destino, codigo) encapsula toda la lógica de envío. Construye el mensaje con MIMEText, configura los headers (Subject, From, To), establece la conexión SMTP con context manager (with), ejecuta STARTTLS, se autentica y envía el mensaje. Retorna True si el envío fue exitoso o False si ocurrió alguna excepción, permitiendo al código llamador manejar el error sin interrumpir el flujo de registro.

8.3 Casos de Uso del Servicio de Correo

- Verificación de cuenta: se envía un código de 6 dígitos al registrarse un nuevo usuario.
- Reenvío de código de verificación: hasta 3 veces por sesión de registro.
- Recuperación de contraseña: se envía un código con expiración de 15 minutos.

8.4 Limitaciones con el servicio SMTP

- Gmail SMTP tiene límites de envío diario (aproximadamente 500 correos/día para cuentas gratuitas). Para escalar, se recomienda migrar a SendGrid, Amazon SES o Mailgun.

- Los correos se envían de forma síncrona, bloqueando la respuesta HTTP hasta completar el envío.
- No se implementa manejo de rebotes ni verificación de entregabilidad.

9. DESPLIEGUE CON DOCKER

9.1 Arquitectura Docker

La aplicación se despliega mediante Docker Compose con los los siguientes servicios que se comunican entre sí en la red de Docker:

Servicio	Imagen	Puerto Expuesto	Función
web	Construida desde Dockerfile local	5000:5000	Servidor Flask de la aplicación Dessert Sacré
db	postgres:15	5432:5432	Base de datos PostgreSQL

9.2 Variables de Entorno del Servicio Web

El servicio 'web' en docker-compose.yml define las siguientes variables de entorno que la aplicación usa:

Variable	Valor en Compose	Descripción
host	host.docker.internal	Host de conexión a PostgreSQL. Permite al contenedor web acceder al servicio db por medio de Docker.
dbname	Dessert_Sacre	Nombre de la base de datos.
user	postgres	Usuario de PostgreSQL.
password	123456	Contraseña de PostgreSQL.

9.3 Persistencia de Datos

El volumen en Docker llamado "postgres_data" persiste los datos de PostgreSQL al iniciar contenedor. Este volumen se monta en /var/lib/postgresql/data dentro del contenedor "db". Si se elimina el contenedor pero no el volumen, los datos se conservan.

9.6 Comandos de Operación

Comandos	Uso
<code>docker compose up --build</code>	Para iniciar todos los servicios en background.
<code>docker compose logs -f web</code>	Ver logs del servicio web en tiempo real
<code>docker compose down</code>	Detener todos los servicios
<code>docker compose down -v</code>	Detener y eliminar volúmenes